Punch Clock
Analyse der IST-Situation

Strukturierung einer Enterprise-Applikation anhand des vorhandenen Quellcodes

Bearbeitung	Zizu11330
Datum	5. November 2026
Vorlage	github.com/zlict/m223-punchclock
Eigener Fork	github.com/Zizu11330/m223-punchclock-Zizu11330
Analysierter Stand	`6fb325c80bf7c917b3d099d63fd0842072122b0d` Branch `main` der Vorlage beim Fork

Zweck und Umfang

Punch Clock ist ein einfaches Zeiterfassungssystem. Im Browser werden Ein- und Austrittszeit eines abgeschlossenen Zeitintervalls erfasst. Eine Tabelle zeigt alle gespeicherten Eintraege mit ID und den beiden Zeitpunkten an.

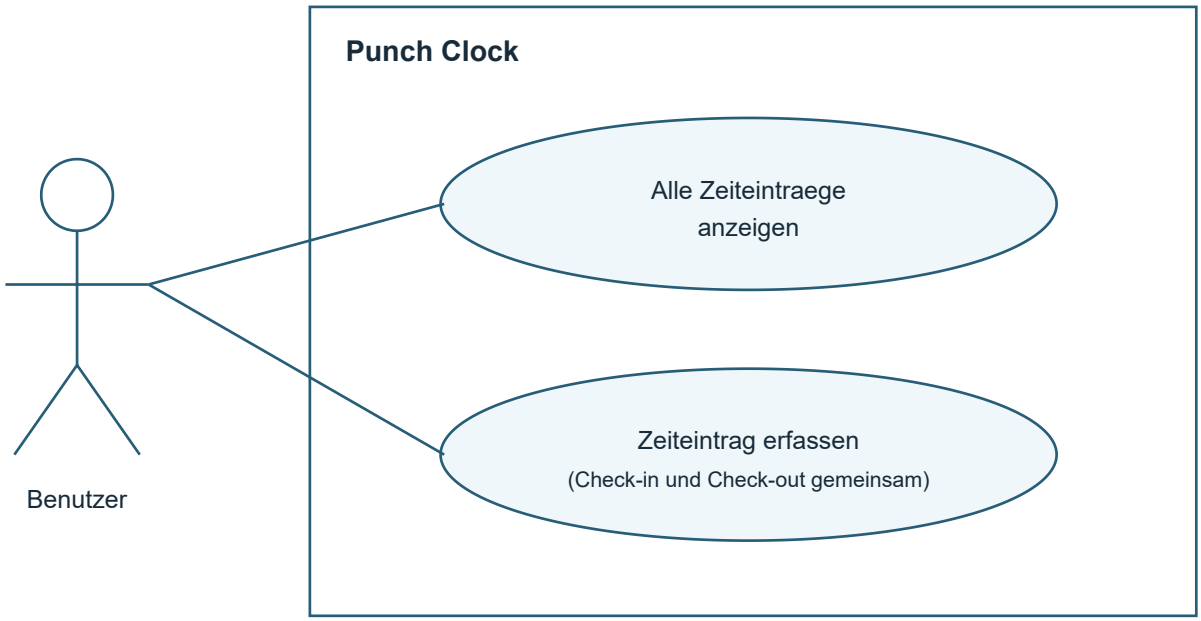
Die Analyse beschreibt ausschliesslich den vorhandenen Stand. Es wurden keine zusaetzlichen Fachfunktionen implementiert. Die Aussagen beruhen auf einer statischen Quellcodeanalyse; sie sind keine Bestaetigung eines erfolgreichen Laufzeittests.

Architektur im Ueberblick

Schicht	Vorhandene Umsetzung
Praesentation	Statisches HTML/CSS und JavaScript; `fetch` fuer JSON-Anfragen.
REST-Schnittstelle	`EntryController` mit GET und POST unter `/entries`.
Anwendungslogik	`EntryService` delegiert Lesezugriffe und kapselt das Speichern in einer Transaktion.
Datenzugriff / Modell	`EntryRepository` nutzt Panache; `Entry` ist eine JPA-Entity. PostgreSQL speichert die Daten.

Technische Basis laut `pom.xml`: Java 25, Quarkus 3.33.4, Maven, Jakarta REST/CDI/Transactions, Hibernate ORM mit Panache und Jackson. OpenAPI dokumentiert die Schnittstelle.

1. UML-Anwendungsfalldiagramm



Akteur und Systemgrenze

Der Benutzer bedient den Webclient. Eine fachliche Unterscheidung zwischen Mitarbeitenden und Administratoren existiert im Code nicht. Die Datenbank ist ein internes technisches Element und deshalb kein fachlicher Akteur.

Anwendungsfall	Ablauf im vorhandenen System
Alle Zeiteintraege anzeigen	Beim Laden der Seite ruft `indexEntries()` GET `/entries` auf. Die Antwort wird in `entries` uebernommen und als Tabelle dargestellt.
Zeiteintrag erfassen	Der Benutzer fuellt zwei Datums- und zwei Zeitfelder aus. `createEntry()` sendet beide Zeitpunkte per POST. Die zurueckgegebene Entity wird an die lokale Liste angehaengt und angezeigt.

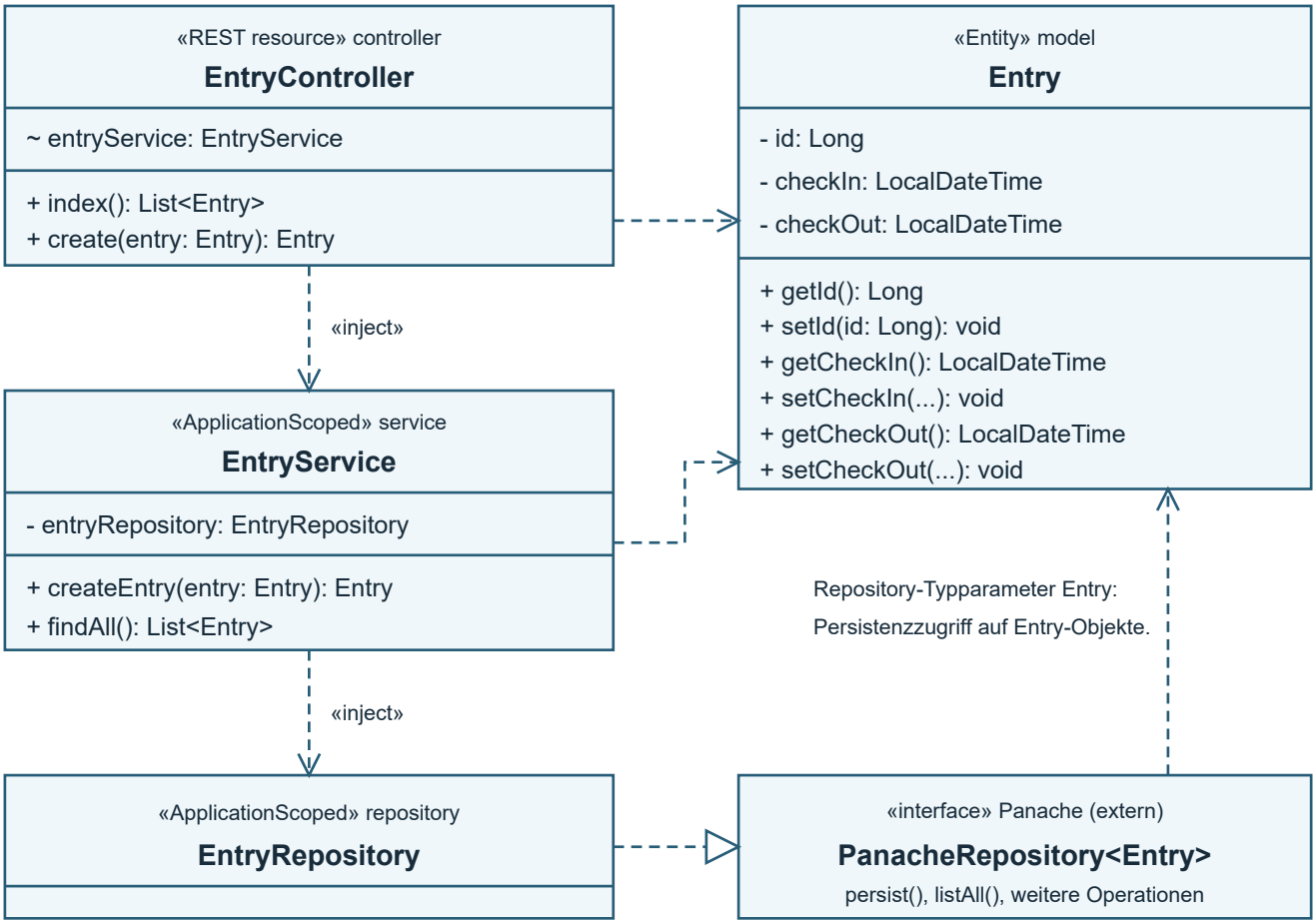
> Kein getrenntes Live-Ein- oder Ausstempeln: Das Formular verlangt Check-in und Check-out bereits beim Speichern.

Nicht Teil der IST-Funktionen

Es gibt keine Anmeldung, Benutzerzuordnung, Rollen, Bearbeitung, Loeschung, Suche, Filterung oder Auswertung der Arbeitsdauer. Diese Funktionen sind weder im Formular noch als entsprechende REST-Endpunkte implementiert.

Belege: `src/main/resources/META-INF/resources/index.html`, `script.js`; `src/main/java/ch/zli/m223/controller/EntryController.java`.

2. UML-Klassendiagramm



Legende: + public, - private, ~ package-private; gestrichelt = Abhaengigkeit.
Hohles Dreieck = Interface-Realisierung; offene Pfeilspitze = Nutzung.

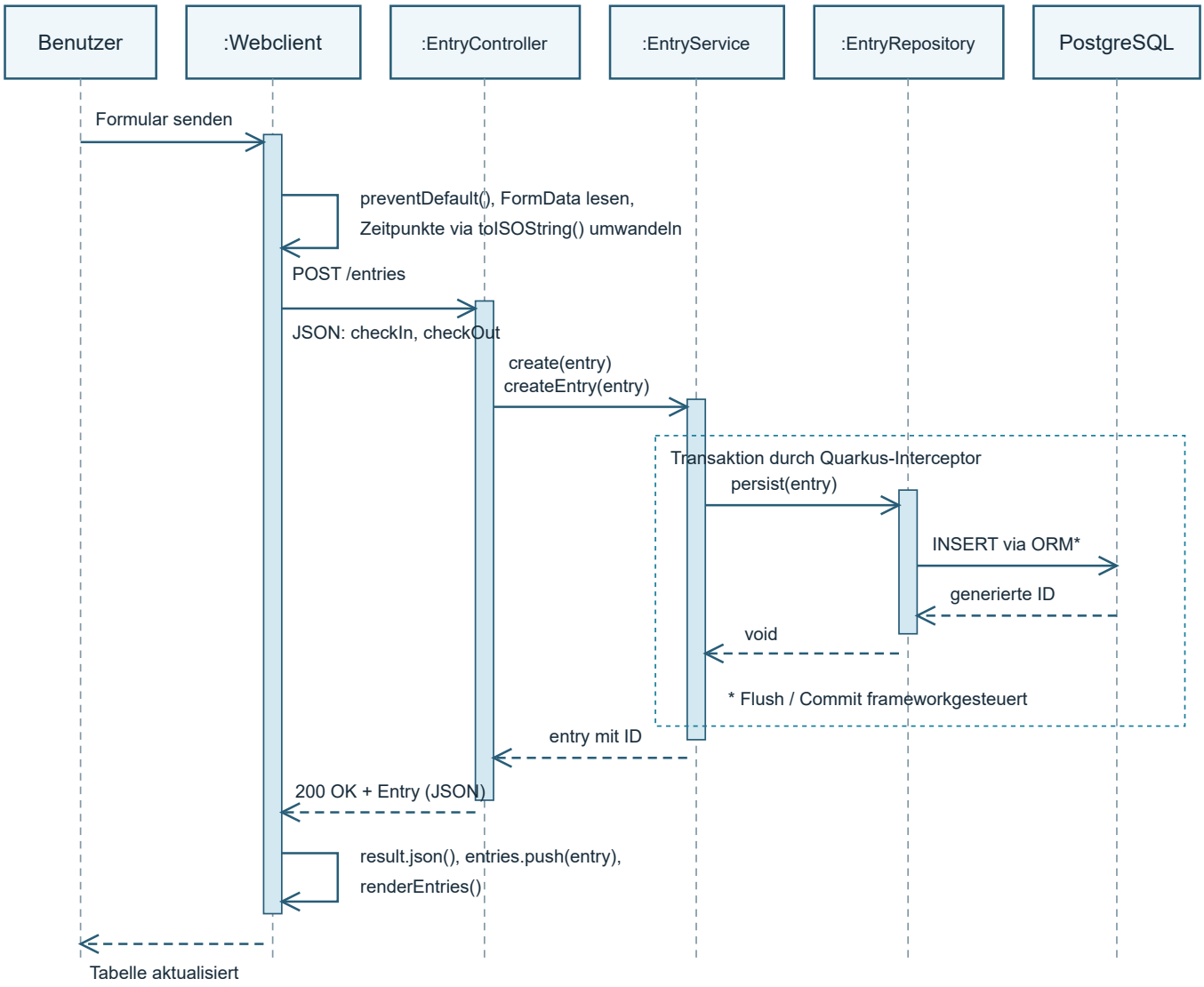
CDI injiziert die Abhaengigkeiten. `EntryRepository` deklariert keine eigenen Methoden, sondern nutzt die Standardoperationen von `PanacheRepository<Entry>`. Die gestrichelten Pfeile zu `Entry` sind Typabhaengigkeiten, keine fachlichen Assoziationen.

`id` ist der automatisch erzeugte Primaerschluessel (IDENTITY). `checkIn` und `checkOut` sind laut JPA-Mapping nicht nullable. Weitere Entities oder Beziehungen gibt es nicht. `createEntry()` ist mit `@Transactional` annotiert; Getter-/Setter-Parameter sind teilweise gekuerzt.

Belege: alle vier Java-Klassen unter `src/main/java/ch/zli/m223`.

3. UML-Sequenzdiagramm

Erfolgsfall: Der Benutzer speichert ein vollstaendiges Zeitintervall.



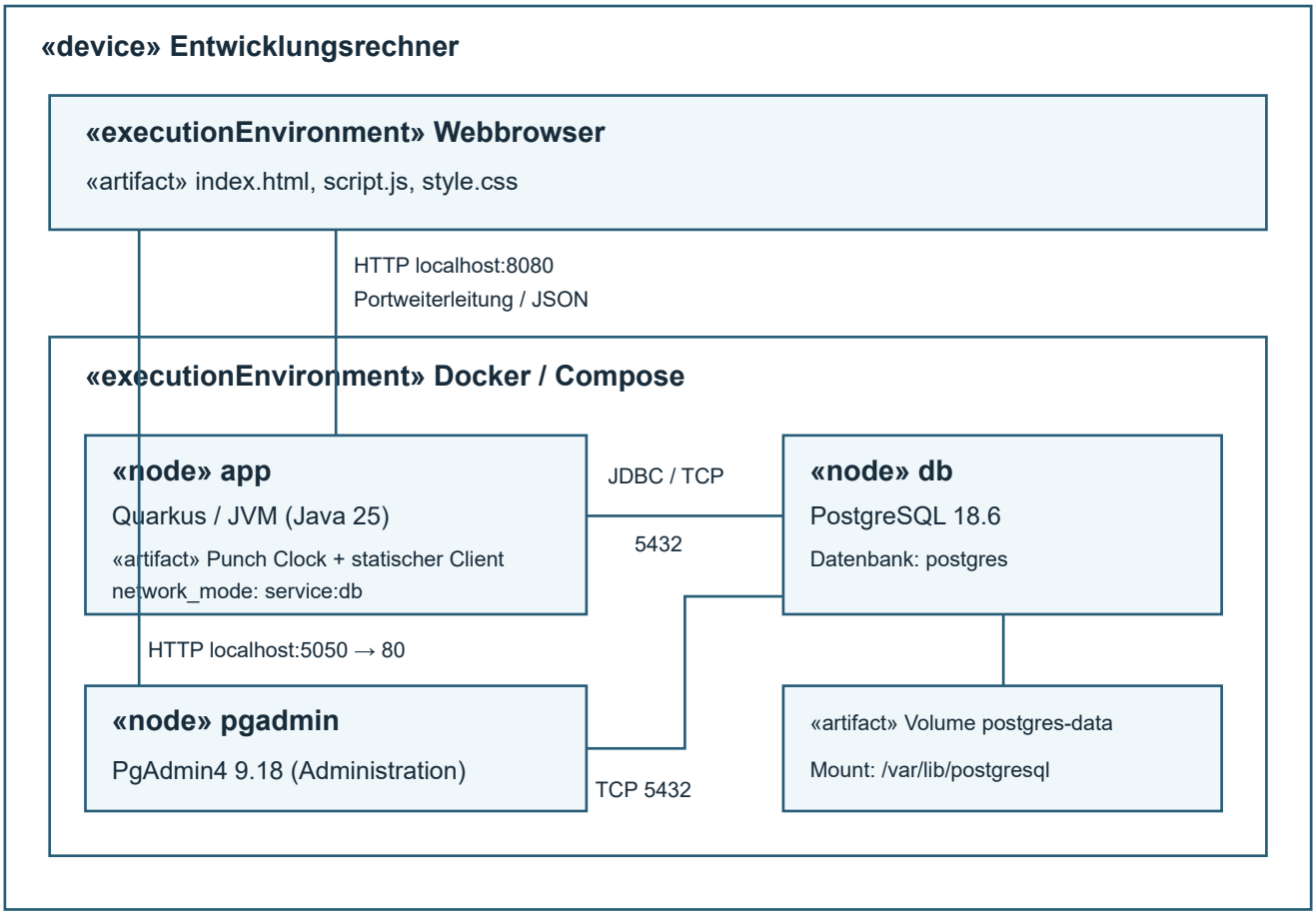
Die Datenbankkommunikation ist vereinfacht dargestellt: Nicht das Repository selbst baut SQL, sondern Hibernate/Panache. Quarkus verwaltet die Transaktion um den Service-Aufruf; ein erfolgreicher Commit liegt vor der erfolgreichen HTTP-Antwort.

Das anschliessende Anzeigen erfolgt lokal, ohne erneutes GET. Der Status `200` entspricht dem vorhandenen Endpoint-Test; ein ausdrueckliches `201 Created` setzt der Controller nicht.

Belege: `script.js`, `EntryController.java`, `EntryService.java`, `EntryRepository.java`, `EntryResourceTest.java`. Fehlerfaelle sind nicht Teil dieses Erfolgsablaufs.

4. UML-Deployment-Diagramm

Konfigurierte Entwicklungsumgebung mit Docker Compose, nicht eine angenommene Produktionsarchitektur.



Quarkus stellt die statischen Dateien sowie die REST-API bereit. Die App verbindet sich gemaess `application.properties` ueber JDBC mit PostgreSQL; Host, Datenbank und Zugang werden als Umgebungsvariablen gesetzt. Fuer das Schema ist `update` konfiguriert.

Die Container `app` und `db` teilen den Netzwerk-Namespace. Port 8080 ist im Devcontainer weitergeleitet. PgAdmin ist separat auf Host-Port 5050 veroeffentlicht und dient nur der Administration. Das benannte Volume bewahrt die PostgreSQL-Daten ausserhalb des Container-Dateisystems auf.

Abweichende Testumgebung

Die Tests verwenden H2 unter `jdbc:h2:mem:test` und `drop-and-create` statt PostgreSQL. Damit wird nicht exakt dieselbe Datenbanktechnik wie in der Entwicklungsumgebung geprueft.

Belege: `.devcontainer/docker-compose.yml`, `.devcontainer/devcontainer.json`, `.devcontainer/Dockerfile`, `src/main/resources/application.properties` und `src/test/resources/application.properties`.

5. Befunde zum IST-Zustand

Aspekt	Beobachtung und Einordnung
Schichtentrennung	Controller, Service, Repository und Entity haben getrennte Aufgaben. Der Controller greift nicht direkt auf die Datenbank zu. Die Entity dient jedoch zugleich als REST-Ein- und Ausgabemodell; separate DTOs fehlen.
Persistenz	Die ID wird von der Datenbank erzeugt. Beide Zeitpunkte sind nicht nullable. Der Service speichert transaktional und verwendet die Panache-Standardmethoden.
Validierung	Das HTML verlangt alle vier Felder. Im Java-Code fehlen Bean-Validation-Regeln sowie eine fachliche Pruefung, dass Check-out nach Check-in liegt. Die Datenbank-Constraints ersetzen diese Pruefung nicht.
Zeitdarstellung	JavaScript wandelt lokale Eingaben mit `toISOString()` in UTC-Zeichenketten mit `Z` um. Das Modell nutzt `LocalDateTime` ohne Zeitzone. Eine explizite durchgaengige Zeitzonenregel ist nicht implementiert; das konkrete Laufzeitverhalten wurde hier nicht getestet.
Fehlerbehandlung	Der Client kontrolliert weder `result.ok` noch HTTP-Fehlerstatus und besitzt keinen `catch`-Handler. Eigene fachliche REST-Fehlerantworten sind in den vier Projektklassen nicht definiert.
Konfiguration	Die API-Basisadresse ist im Client fest auf `http://localhost:8080` gesetzt. Ein Betrieb unter anderer Adresse erfordert eine Anpassung. CORS ist aktiviert; daraus folgt allein keine Aussage ueber alle erlaubten Origins.
Testumfang	`EntryResourceTest` beschreibt GET auf leerem Bestand, POST eines gueltigen Eintrags (200) und anschliessendes GET. `EntryResourceIT` erbt diese Tests fuer den paketierten Modus. Negative Faelle und Browserinteraktionen sind nicht enthalten; `skipITs` ist standardmaessig true und im Native-Profil false.

Fazit

Die Vorlage ist eine kleine, nachvollziehbar geschichtete Grundlage fuer das Modul 223. Sie zeigt den gesamten Weg von einer Browseraktion ueber REST und Anwendungslogik bis zur relationalen Persistenz. Fachlich ist sie auf das Erfassen und Auflisten abgeschlossener Zeitintervalle begrenzt.

Die aufgefuehrten Luecken beschreiben den Ausgangszustand und sind keine in dieser Aufgabe umgesetzten Erweiterungen. Fuer eine spaetere Weiterentwicklung waeren vor allem fachliche Validierung, eine klare Zeitkonvention und eine sichtbare Fehlerbehandlung zu klaeren.

Quellen und Methode

Primaerquelle: Quellcode der Vorlage am dokumentierten Commit. Ausgewertet wurden die vier Java-Produktivklassen, Webclient-Dateien, beide Testklassen, beide Anwendungskonfigurationen, `pom.xml`, Devcontainer-Konfiguration und README. Alle UML-Diagramme wurden aus diesen Dateien abgeleitet. Durchgezogene Linien im Anwendungsfall- und Deployment-Diagramm zeigen Assoziationen bzw. Kommunikationspfade. Die HTML-Quelle enthaelt alle Diagramme lokal als editierbares SVG.